

Deep Dive: Creating Your First Ontology

Palantir Foundry 本体论深度课程 · 按左侧导航顺序完整收录 (20 个 lesson, 含图片)

抓取日期: 2026-09-07 · 来源: learn.palantir.com

课程总览

课程名称: Deep Dive: Creating Your First Ontology

来源链接: <https://learn.palantir.com/deep-dive-creating-your-first-ontology>

目录（按左侧导航顺序）

1. Introduction
2. Permissions & Help
3. Install the Marketplace Bundle
4. Context: Fresh Air
5. Object Explorer
6. Fresh Air Operations
7. Reflections
8. Context: Why data lineage matters
9. Data Lineage: Fresh Air pipeline
10. Data Lineage Exploration
11. Reflections
12. Context: Extending Fresh Air's Ontology
13. Ontology Manager
14. Create an object type
15. Create a link type
16. Create an action type
17. Execute your action
18. Reflections
19. Conclusion
20. [Optional] Course Feedback

Introduction

Lesson 1 / 20 · [原文链接](#)

The Ontology

The data that is used most regularly in Foundry is presented as **objects**. For example, you can open a specific Airline object, Aircraft object, or Customer object and see the essential data (that you have permission to see) about that particular real-world thing. This object's data is typically aggregated from many different original data sources. For example, an Aircraft object might contain information pulled from purchase records, maintenance systems, flight systems, and more.

The **Ontology** defines the set of **object types** available. Each instance of Foundry has its own unique set of object types that are specific to that organization and the problems they face. For example, an airline or a military organization would typically have an Aircraft object type, whereas a pharmaceuticals company would not. Furthermore, the Aircraft object type might have different properties and different relationships to other object types at an airline than it does for the military. In this way, the Ontology reflects the real-world objects that users at a given organization think about and operate on every day.

The Value of the Ontology

The Ontology offers several benefits:

- It makes data easier to use.
- It provides a common operational vocabulary across workflows and across the organization.
- It supports operational decisions through well-defined actions.
- It powers AI solutions in an operational day-to-day context.

This Course

In this course, you will:

- Get hands-on with an existing Ontology to better understand what it is and how it makes data easier to use.
- See how an Ontology is built and how it provides a common vocabulary to an organization.
- Learn how to start building your own Ontology by defining your own object type, relationship, and action
- Learn more about how the Ontology lies at the heart of Foundry and how that position impacts how you should think about Foundry-based solutions

Prerequisites

This course assumes that you have completed [Speedrun: Your First End-to-End Workflow](#).

While you will not reuse any of that work here, you should be familiar with the concepts covered in the E2E Speedrun. Specifically, this Deep Dive: Ontology course assumes you are already familiar with the basics of:

- ingesting data into Foundry

- transforming and cleaning data using Pipeline Builder
- creating your first object type
- using objects to build a Workshop application

Our partners at Ontologize have developed [a step-by-step YouTube video](#) of this course. Feel free to use the video to follow along as you complete the course material.

Permissions & Help

Lesson 2 / 20 · [原文链接](#)

As you work through this course, you may encounter certain issues. For example, there may be specific features, limitations, or security controls in place on your Foundry instance that prevent you from following these generic instructions.

Depending on the issue that you encounter, here is how to get help!

Training Application

Your organization's Foundry Platform Administrator may have posted local documentation within Foundry's **Training** application. This will typically cover any user restrictions that exist in your specific Foundry enrollment regarding creating projects, uploading data, creating or editing objects, or deploying Marketplace bundles. It may also include organization-specific training content.

You can open the Training app from the bottom of Foundry's left navigation bar by selecting **Support > Training**.

Permissions

To complete this specific learning course, you will need access to the following platform components:

- Marketplace
- Object Explorer
- Data Lineage
- Ontology Manager

You will need the following permissions in order to run all steps of this learning course:

Permissions Required	Workaround if permissions are not available
Deploy resources from Marketplace	Platform Admin team will need to deploy resources from Marketplace for every user completing the training.
Create Ontology assets (Object Types, Links and Actions)	None. Permissions are required.

Documentation

You can find out more about any of the Foundry applications or concepts presented in this course by searching in the Foundry documentation: <https://www.palantir.com/docs/foundry>.

Screenshots and Instructions

Our platform is evolving quickly. The screenshots in this course may differ slightly from how Foundry appears on your screen. In most cases, these differences should be minor and cosmetic, and it should still be possible to continue with the course. If you notice such differences or are unable to continue because of them, please report them to training-feedback@palantir.com.

Feedback

This training is being actively developed and we'd love to hear your feedback! If you have comments or suggestions, or if you encounter issues you cannot resolve by any of the means listed above, please reach out to training-feedback@palantir.com.

Install the Marketplace Bundle

Lesson 3 / 20 · [原文链接](#)

This course requires a specific Marketplace bundle to be installed to provide the necessary data, pipelines, and object types.

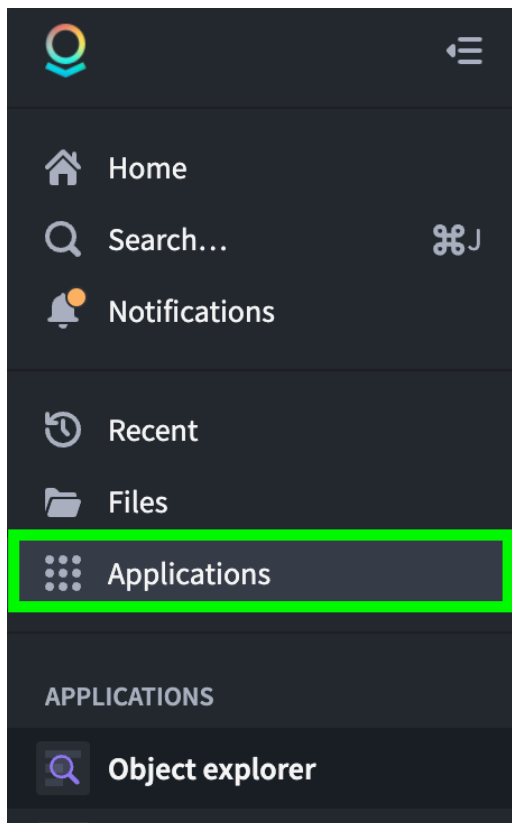
- If someone else has already installed it on your Foundry instance, you can likely reuse it.
- Otherwise, you will need to install your own copy.

Having trouble?

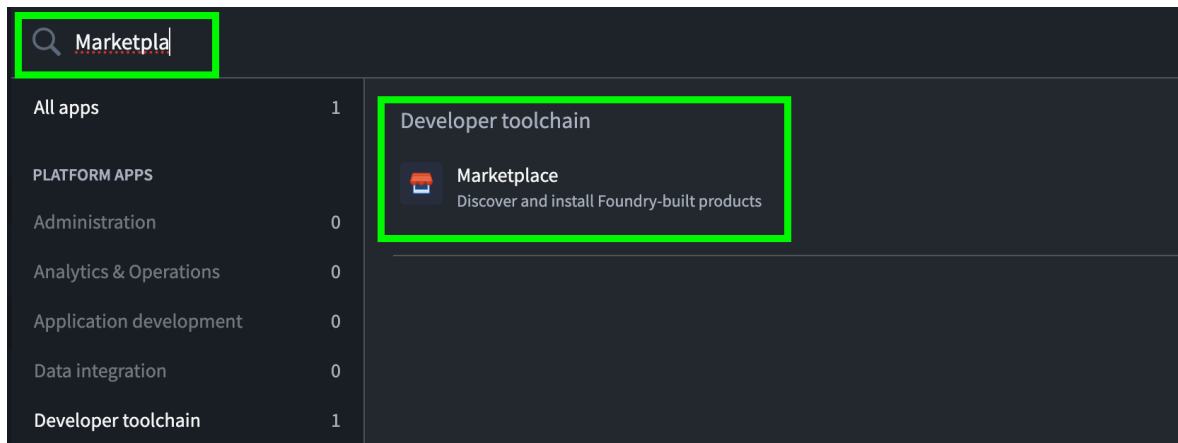
If you're having trouble installing Marketplace products, your organization may be controlling which users are able to install Marketplace products or create new resources such as Projects, Objects, Actions, etc. Please check your local Training app or reach out to your platform administrator for help. See the earlier *Permissions & Help* section for details.

Step 1: Open Marketplace

1. Click on *Applications* on the left side of the screen

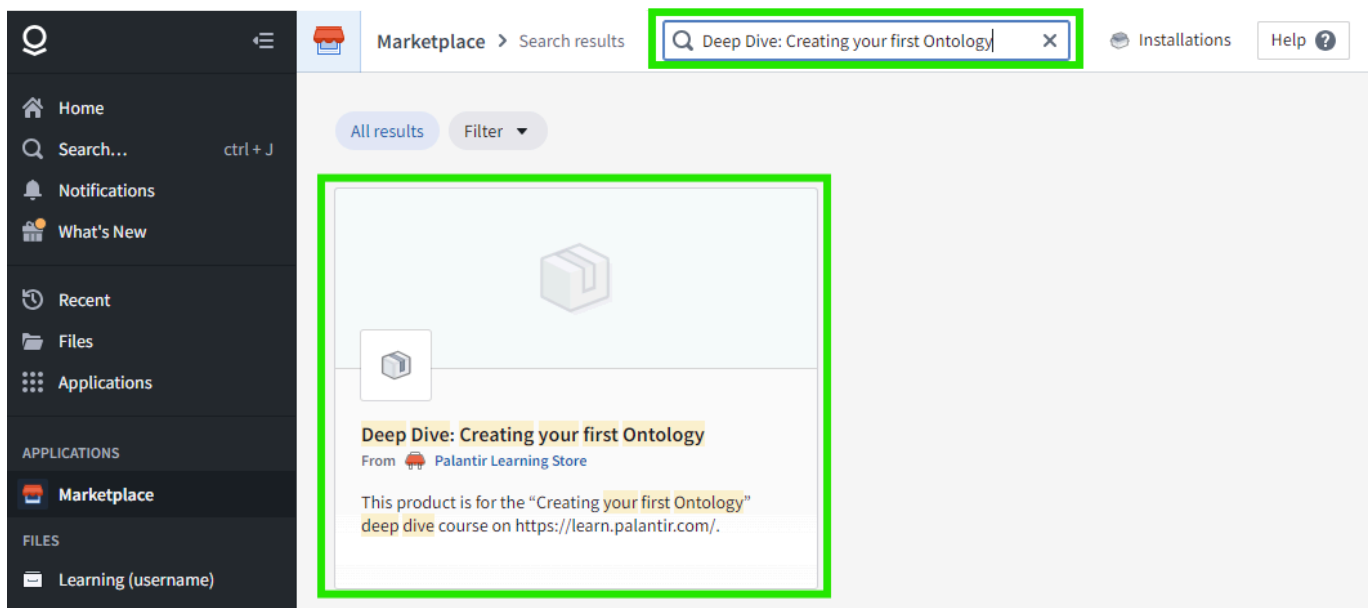


2. Search for *Marketplace* and open it.



Step 2: Open *Deep Dive: Creating Your First Ontology*

1. Search for the **Deep Dive: Creating Your First Ontology** product in the top search bar labeled *Search products...* (not the lower search bar labeled *Search stores...*)
2. Click into the corresponding product



You can then check if there are any installations listed on the right side of the screen:

 Marketplace > > >  Deep Dive: Creating Your First Ontology

  Search products in store...

 Installations

Help 

 Back to store

 **Deep Dive: Creating Your First Ontology** 1.1.0 

Open 

Install again

This product contains datasets and objects for the Deep Dive: Building Your First Ontology course on learn.palantir.com.

Overview

Changelogs

Content

Inputs

Installations

Linked products

Description

This product contains a set of pre-built pipelines and open-source datasets that back a robust ontology in the airline domain.
This product is for the “Creating your first Ontology” deep dive course on <https://learn.palantir.com/>, which guides you through the value of Foundry’s ontology and how to create your object, link, and action types.
(The notional healthcare data that supported the original version of this course is still included in a v0 folder.)

Content

See all

Workshop Applications (21)

 Panel Object View Module f6d9db44-300f-4d77-adce-c8942eafc463

 Object View Module 8d71d537-e672-4fd0-b8da-d86ace9eb51a

Version1.1.0

Release ChannelRelease

Store Palantir Learning ...

Installations (1)

 Deep Dive: Creating Your First Ontology - username

Upgrade available 1.0.0

Installed in  Deep Dive: Creating your first ...

See all

If there are any existing installations: You can skip the rest of this lesson and try using one of these installations starting with the *Object Explorer* section of this course.

If there are no installations OR if you later run into ontology permission issues in this course: Continue below with these instructions to install your own version of the course resources.

Step 3: Start the installation

1. Click *Install* or *Install again* in the top right corner


Marketplace > ... > Deep


In  Search products in store...

 Installations

Help ?

← Back to store


Deep Dive: Creating your first Ontology
0.3.0

Install

This product is for the “Creating your first Ontology” deep dive course on <https://learn.palantir.com/>.

Overview
Changelogs
Content
Inputs
Installations
Linked products

Description

This product contains a set of pre-built pipelines and synthetic datasets that serve as the starting point for constructing a robust ontology in the healthcare domain.

This product is for the “Creating your first Ontology” deep dive course on <https://learn.palantir.com/>, which guides you through the process of creating and linking object types such as Patient, Surgery, Treatment Type, and `Patient Outcome`.

Version
0.3.0

Release Channel
Release

Store
 Palantir Learning ...

Content
See all

 Datasets
4

 Pipelines
1

Step 4: Customize the installation

- Set your username as the installation suffix
 - Toggle on *Installation suffix*
 - Enter your own `<username>`
- Change the installation location to your course-specific training folder
 - Click *Use existing location*
 - Click *Browse*
 - Browse to your *Learning (<username>)* project
 - If you do not have one yet from completing other tutorials, click *Create new project* and create a project named `Learning (<username>)` where `<username>` is your own username.
 - Create a new `Deep Dive: Creating your first Ontology` folder
 - Click *Save*

3. Click *Next*

The screenshot shows the 'General' configuration page for installing a product. The left sidebar indicates the product is in 'Draft' status (version 0.3.0) and lists sections: General (selected), Content, and Review. The main area is titled 'General' and contains several sections:

- Installation mode:** Set to 'Bootstrap'. A note states: 'The recommended and default settings for this installation mode specified by the product builder.'
- Installation suffix:** A toggle switch is turned on. A note explains: 'The installation name will be generated with a suffix after the product name. The suffix will remain the same if the underlying product name is changed later.' Below this, the text 'username' is entered in the suffix field. A 'Preview' button shows the resulting name: 'Deep Dive: Creating your first Ontology - username'.
- Installation location:** The 'Install location' is set to 'Deep Dive: Creating your first Ontology' (highlighted with a green box). A 'Change' button and a 'Generate new project' button are also present. Below this, the 'Namespace' is set to '-19dedf'.
- Permissions:** Under 'Roles', a message states: 'Role grants can only be set up when creating a new project through marketplace'. Under 'Organizations', a checkbox is checked.

A 'Next' button with a right arrow is located at the bottom right of the configuration area.

Step 5: Confirm the content

1. Decide whether to prefix ontology entity names
 1. If there are many existing installations, you can *Prefix ontology entities* with your username.
 2. If this is the first installation of this product, you can skip this.

2. Click *Next*

Step 6: Install

1. Ensure that all validations have passed
2. Click *Install*

**Marketplace** > ... > Draft 1.1.0

 Search products...

 Installations

 Help ?

 Deep Dive: Creating Your First Ontology
- username

Installing version 1.1.0 Draft ...

General

Content

Review

Review

✓ All validations have passed

Messages (1) ^

 Automatic upgrades will be disabled for this installation

Compass resources

Workshop Application (21) v

Pipeline (32) v

Dataset (51) v

Notepad document (5) v

File (2) v

Data health check

Data health check (5) v

Object Explorer edits

Object view (7) v

Ontology edits

Object type (8) v

Install

Step 7: Complete the installation

1. Wait for the installation to finish
 - It generally takes 5 to 8 minutes to install resources
 - It then takes 5 to 10 more minutes to build them

- And potentially another 10 minutes or so (not shown in the UI here) to finishing indexing all object data into the object layer

Context: Fresh Air

Lesson 4 / 20 · [原文链接](#)

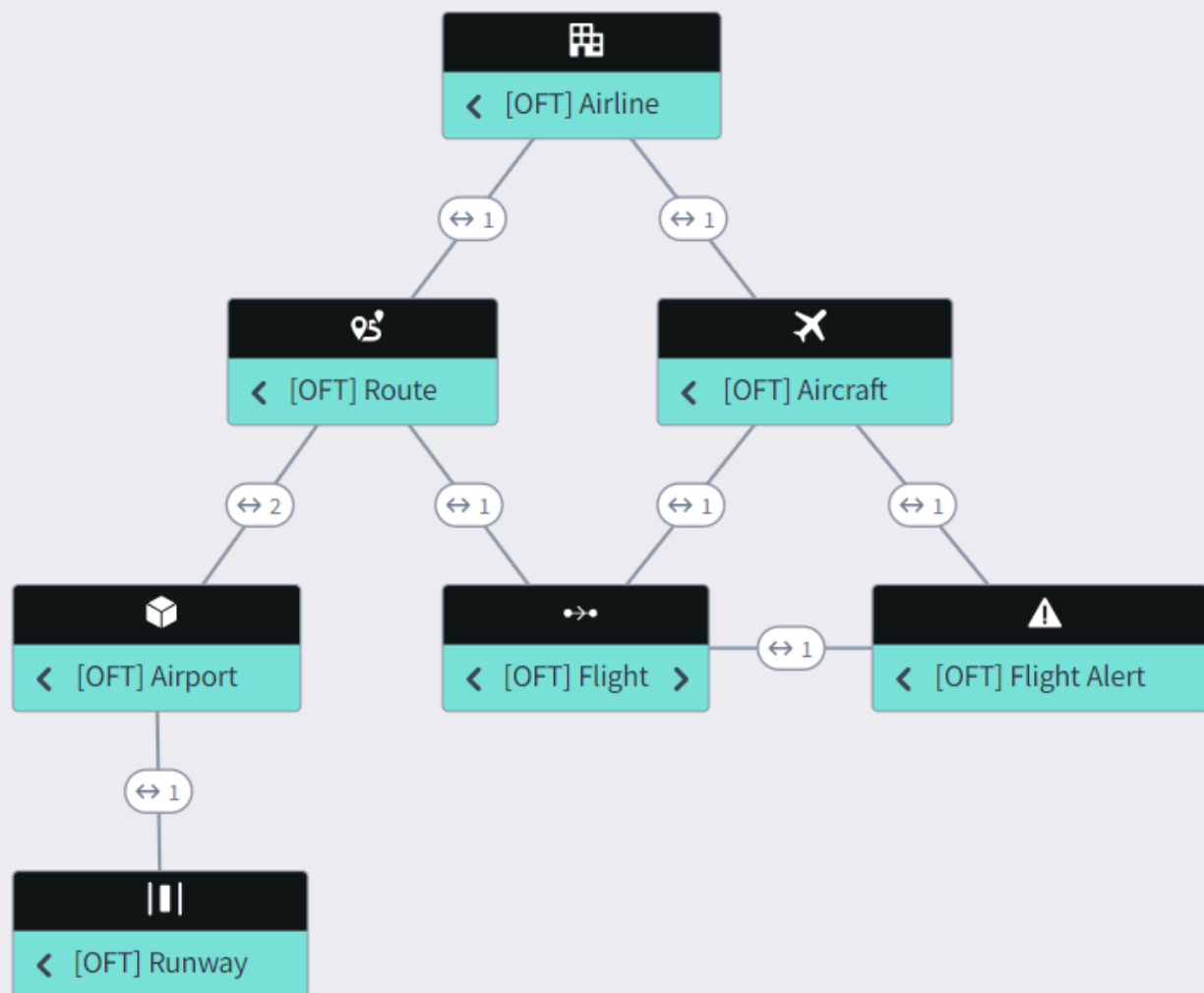
Fresh Air

For the the following exercise, assume that you are a new employee at Fresh Air. Fresh Air is a (notional) regional airline of approximately 4,500 employees that services the eastern half of the United States. They currently fly approximately 500 flights a day to 83 destinations. Although technically headquartered in Fort Worth TX, their busiest hubs are all on the east coast.

To start, assume you have just joined the Flight Operations department. Later in this course, we'll see what the same data looks like from the perspective of other roles and departments.

Ontology

As a new employee at Fresh Air, you are not familiar with all of their data systems yet. However, from your new hire orientation, you know some of the basic terms of the industry. These types are present in Fresh Air's Ontology:



In summary, an **Airline** operates a fleet of **Aircraft**. These Aircraft regularly fly from one **Airport** to another, where they land or takeoff from a given **Runway**. The connection between any two Airports along which Aircraft regularly fly is a **Route**. A **Flight** is a specific instance of an Aircraft flying along a Route at a particular date and time. Occasionally, a Flight might encounter an issue, such as delay, cancellation, or even a diversion to another airport that is not normally part of the route. These non-standard events generate a **Flight Alert**, which the Flight Operations team then responds to.

The data we will explore using this Ontology includes details from all US airlines, not only from Fresh Air. Because these are common object types beyond the bounds of this specific course, the object types include an **OFT** (Ontology Foundry Training) prefix to avoid any naming conflicts with any similar types found in your ontology.

Object Explorer

Lesson 5 / 20 · [原文链接](#)

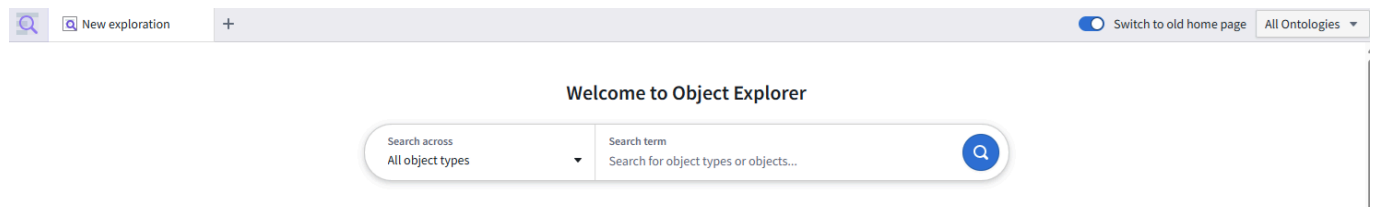
There are many Foundry applications that let you work with objects and the Ontology. We will start with the simplest of them, which is Object Explorer. **Object Explorer** is used for object searches and for basic exploration and analysis across sets of objects.

Step 1: Open Object Explorer

1. Open **Object explorer**

- You will find this in the Applications menu in the left side bar

You should then see something like this:



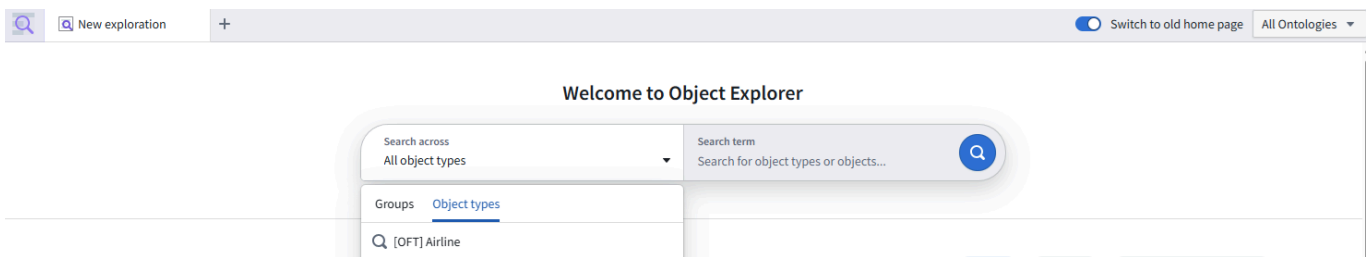
Fresh Air Operations

Lesson 6 / 20 · [原文链接](#)

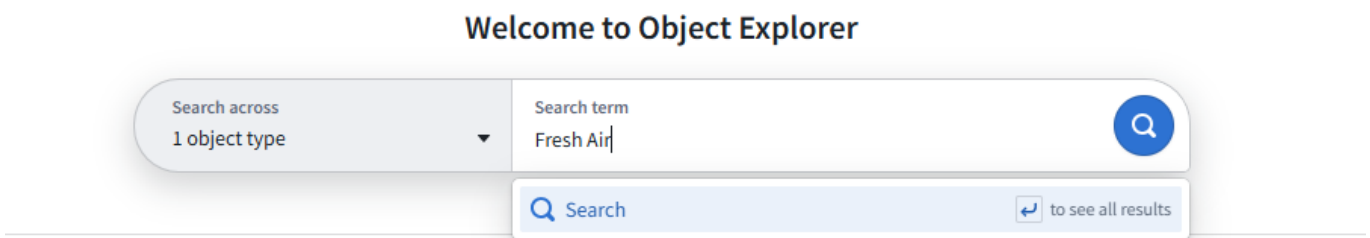
To start, find and open the **Fresh Air Inc** object. Its object type is **[OFT] Airline**. Because there may be a lot of data and many object types in a given Foundry instance, this search will be more reliable if you filter down to the object type first.

Step 1: Open Fresh Air Inc

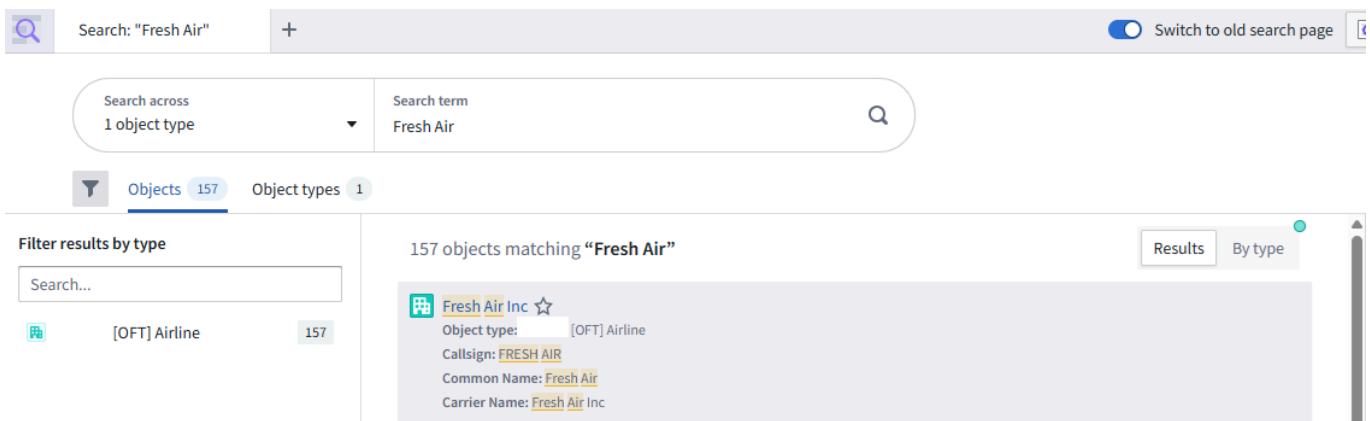
1. Change the **All** drop-down at the left of the search bar to the object type **[OFT] Airline**



2. Search for **Fresh Air**



3. Click on the **Fresh Air Inc** object in your search results

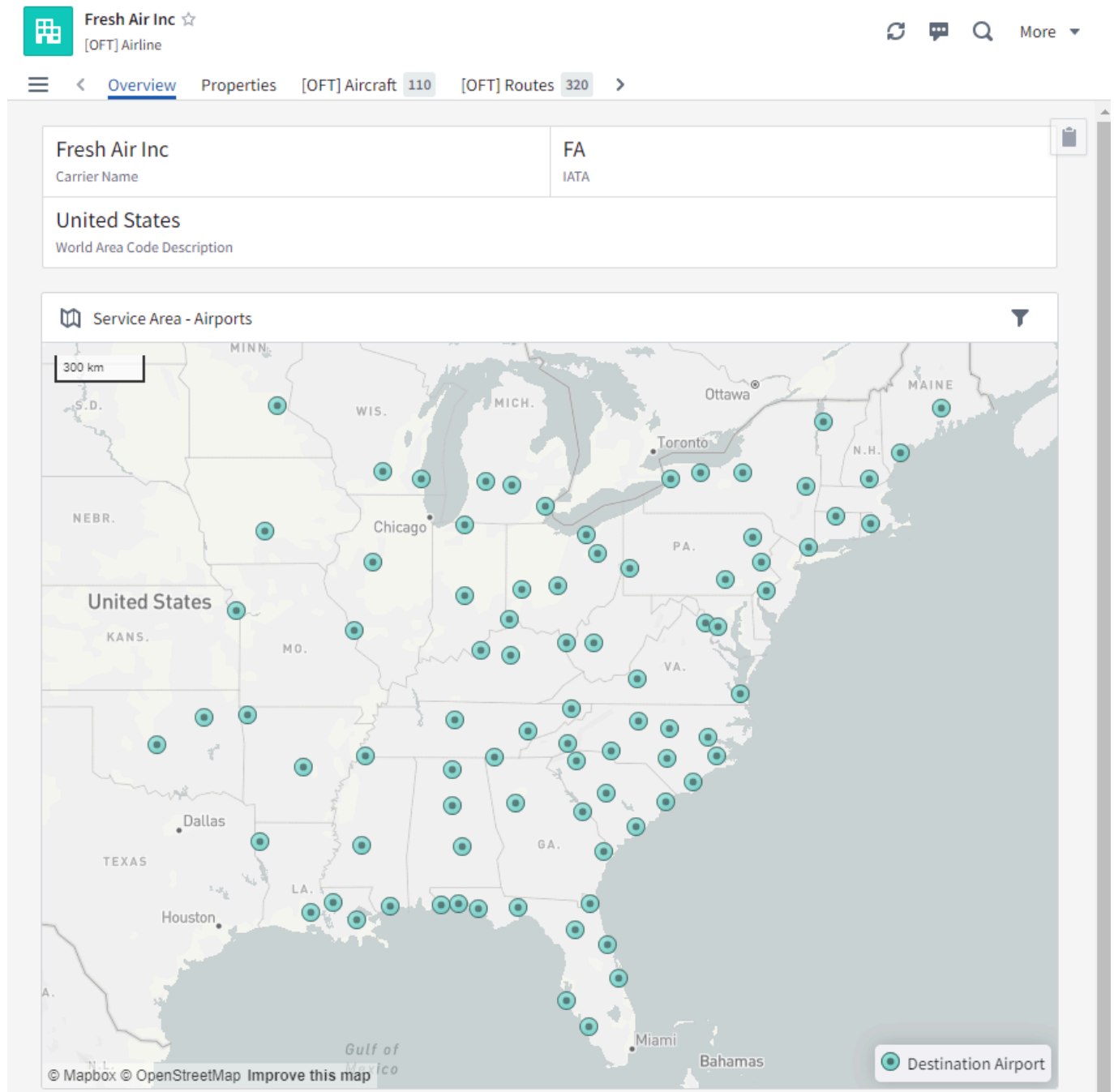


Help: I can't find these object types!

The necessary marketplace bundle must be installed on your Foundry instance, the installation

needs to be fully built to create all the necessary objects, and you then need permission to the see those objects and their data. See the previous *Getting Started* section for more.

This should open an object that looks like this:



This is an example of a single Foundry object. Besides property and links, the view of a object can be augmented with additional derived properties and visualizations. Note that this object view includes multiple tabs along the top and links to additional objects.

Single Object

Using only the **Fresh Air Inc** object in Object Explorer, answer the following questions. Highlight the space after each *Hint* or *Answer* to reveal them.

What is Fresh Air's IATA code?

Hint:

Answer:

What is Fresh Air's ICAO code?

Hint:

Answer:

What does ICAO stand for?

Hint:

Answer:

Who is the single manufacturer for all of Fresh Air's aircraft? Fresh Air has a fleet of 110 aircraft. While it has two different models of planes, all of those planes come from a single manufacturer.

Hint:

Answer:

Fresh Air's Routes

So you can learn a lot of from a single object, but you can learn much more by traversing an object's links and by exploring object sets.

Step 2: View Fresh Air's Routes in a new search

1. Start with the **Fresh Air Inc** object
2. Switch to the **Routes** tab
3. Hover over the top of the list of Routes and click **Open in Explorer**

Search: "fresh air" × Fresh Air Inc × +

Fresh Air Inc ☆
[OFT] Airline

Overview Properties Flights Routes

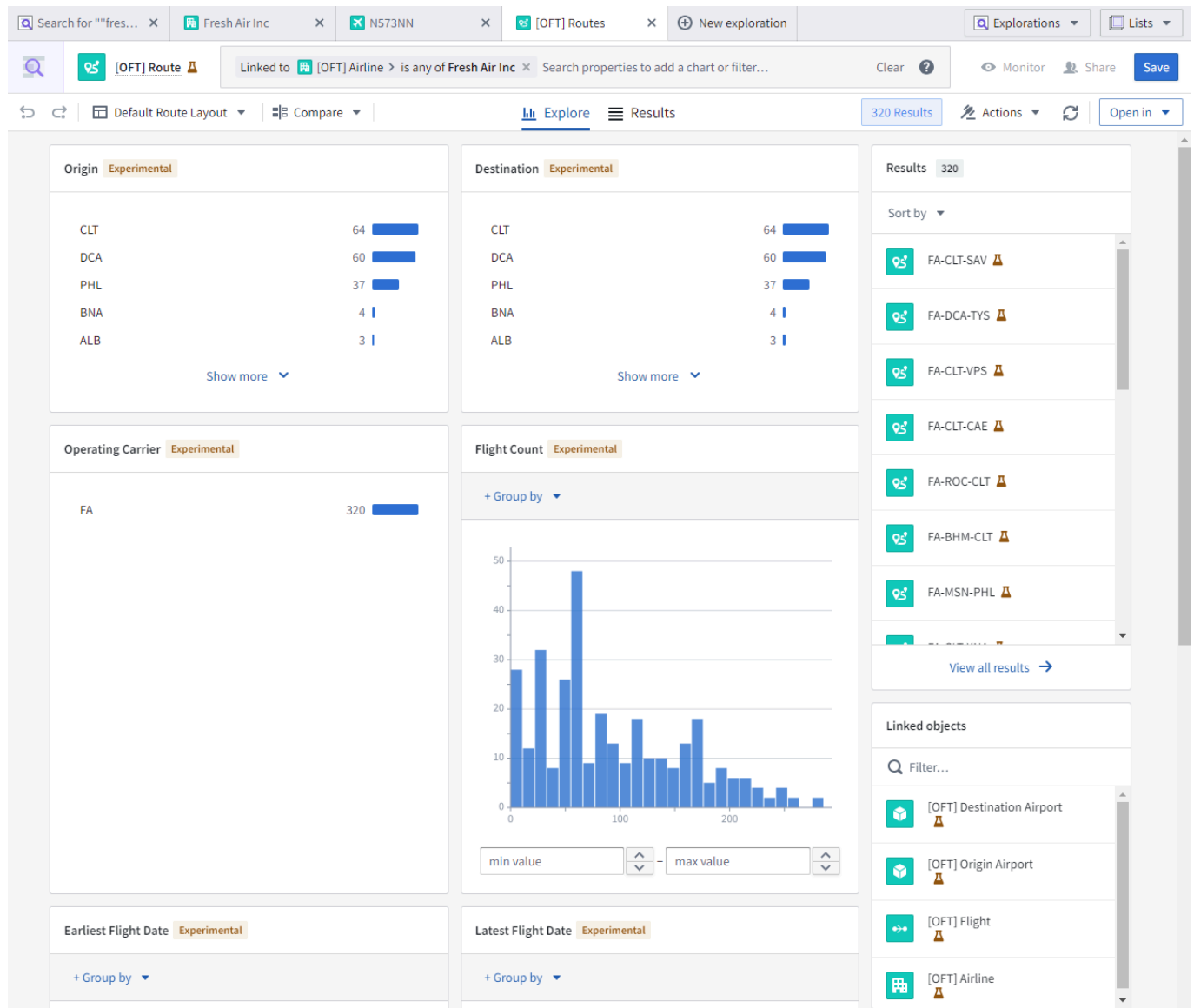
Open in Explorer

[OFT] Route 320

- > FA-CLT-ILM
- > FA-CLT-TUL
- > FA-TUL-CLT
- > FA-AVL-DCA
- > FA-CAK-CLT

Viewing 5 of 320 [Show more](#)

You should now see an exploration of these **320 routes**:



The results count is listed in the top right of the screen.

Explorations

An *exploration* lets you explore a set of objects through a variety of charts. Each chart is derived from one of the properties of that object type. Some properties are charted by default, but it is possible to edit which properties are charted, reorder the charts, and add new charts.

As suggested by the Route titles listed in the *Results* panel on the right, a Route object is an origin-to-destination path, as serviced by a particular airline. For example, FA-CLT-ALB is the FA (Fresh Air) route from CLT (Charlotte, NC) to ALB (Albany, NY).

To get comfortable with explorations and the kinds of customizing you can do, answer the following questions. They can all be answered using only the **Origin** chart.

If you happen to accidentally delete a chart while answering these questions, click the *Undo* icon in the top left of the screen.

Which three airports together serve as the origin for the greatest number of Fresh Air routes? That is, what are top 3 origins for Fresh Air routes?

Answer:

How many routes originate from the DAY airport?

Hint:

Answer:

Don't forget to clear any filters you've applied to the Origin chart before answering the next question.

What percentage of Fresh Air's routes originate from CLT?

Hint:

Answer:

For this last question, you'll need to combine a couple charts. Don't forget to clear any filters first.

Are there any Fresh Air routes that don't involve one of the top 3 airports as either an origin or destination?

Hint:

Answer:

Traversing Links

When you are exploring an object set, you can use the **Linked objects** entries in the lower right to switch to all of the linked objects according to some relationship link. For example, suppose we're starting with all 320 of Fresh Air's Routes. You could then switch to all of the Origin Airports for those Routes, or you could switch to all of the Flights that occur along those Routes.

For example, let's see how many Origin Airports are serviced by Fresh Air.

Step 3: Traverse links to the set Fresh Air's origin airports

1. Confirm that you are starting with 320 Results (in the top right of the screen) for Fresh Air's Routes
 1. Clear any other filters you have.
2. Click **[OFT] Origin Airport** in the lower right

Your set of charts should change because you are now looking at a new object set of Airports (not Routes).

This process of traversing links from one object set to another is often called *pivoting* from one object set to another. (If you are already familiar with pivot tables, I recommend treating this as an unrelated concept with the same name. Trying to think of link traversals in terms of pivot tables will only lead to more confusion than if you start with a fresh mental model of the process as described above.)

You can confirm that Fresh Air has the same number of **Destination Airports**.

Step 4: Traverse links to the set Fresh Air's destination airports

1. Undo the pivot that you just did by clicking the *Undo* arrow in the top left of the screen
 2. Click **[OFT] Destination Airport** in the lower right
-

Challenge Puzzles

If you'd like more practice, here are some optional questions to answer about Fresh Air's operations:

What is the highest elevation origin airport that Fresh Air services? Does it have a runway of at least 7,000 feet?

Hint:

Answer:

What is the Fresh Air origin airport that is the located the farthest west?

Hint:

Answer:

Reflections

Lesson 7 / 20 · [原文链接](#)

Hopefully this experience of using Object Explorer has shown how **easy** it is to work with Foundry data in real-world terms. Without searching for any backing tables, thinking about join keys, or using a query language, you were able to quickly learn a great many details about Fresh Air's operations, such as:

- Their IATA and ICAO codes
- The make-up of their aircraft fleet (2 models of Bombardier aircraft)
- Although Fresh Air services 84 different airports, nearly all of their routes start or end in one of three three hub airports (CLT, DCA, and PHL). The only exception is the BNA-RDU/RDU-BNA route.
- If you completed the challenge puzzles, you were able to find highest airport and confirm that it has a one runway that is long enough to accommodate Fresh Air's aircraft.

Object Explorer is a quick way for any user, regardless of technical background, to get started using objects. Using only point-and-click controls, you can search for single objects of interest, follow links to related objects, and explore a single object set in detail.

Object Explorer is only one place where objects are used in Foundry.

- For more complex object-based analysis—such as doing richer numerical analysis, building visualizations, deriving timeseries, comparing multiple object sets, or building interactive reporting dashboards--you should consider [Quiver](#) instead.
- On the other hand, if you have a regular operational workflow, building a dedicated [Workshop](#) application (as you did as part of the earlier End-to-End Speedrun course) will provide users a smoother and more tailored tool than relying on Object Explorer alone.

Now that you have some idea why Fresh Air's ontology is useful to individual end users, let's see why it's valuable to data engineers and the organization as a whole.

Context: Why data lineage matters

Lesson 8 / 20 · [原文链接](#)

With many people working within the same platform, it can get difficult to track the current state of data. Their work might overlap without them actually speaking to each other.

A data engineer might build a pipeline to clean up the data brought in from an external system. A data analyst might augment that clean data with manually uploaded data from a different data source to build out a view specific to her use case. A year later, another colleague continues that data analyst's work in light of a new business focus.

As layers and layers of this collaborative work happen over years with a rotating cast of users, the current state of any given dataset and its relationship to others in Foundry could become very confusing.

The Data Lineage tool is intended to provide a bird's-eye view of how datasets, object types, and certain resources are connected to each other in Foundry. You can also learn a lot about the state of each dataset, as well as take action on them (such as rebuild a dataset that is out-of-date with its ancestors/inputs).

While Data Lineage is mostly often used by data engineers, it can be useful to any role in Foundry.

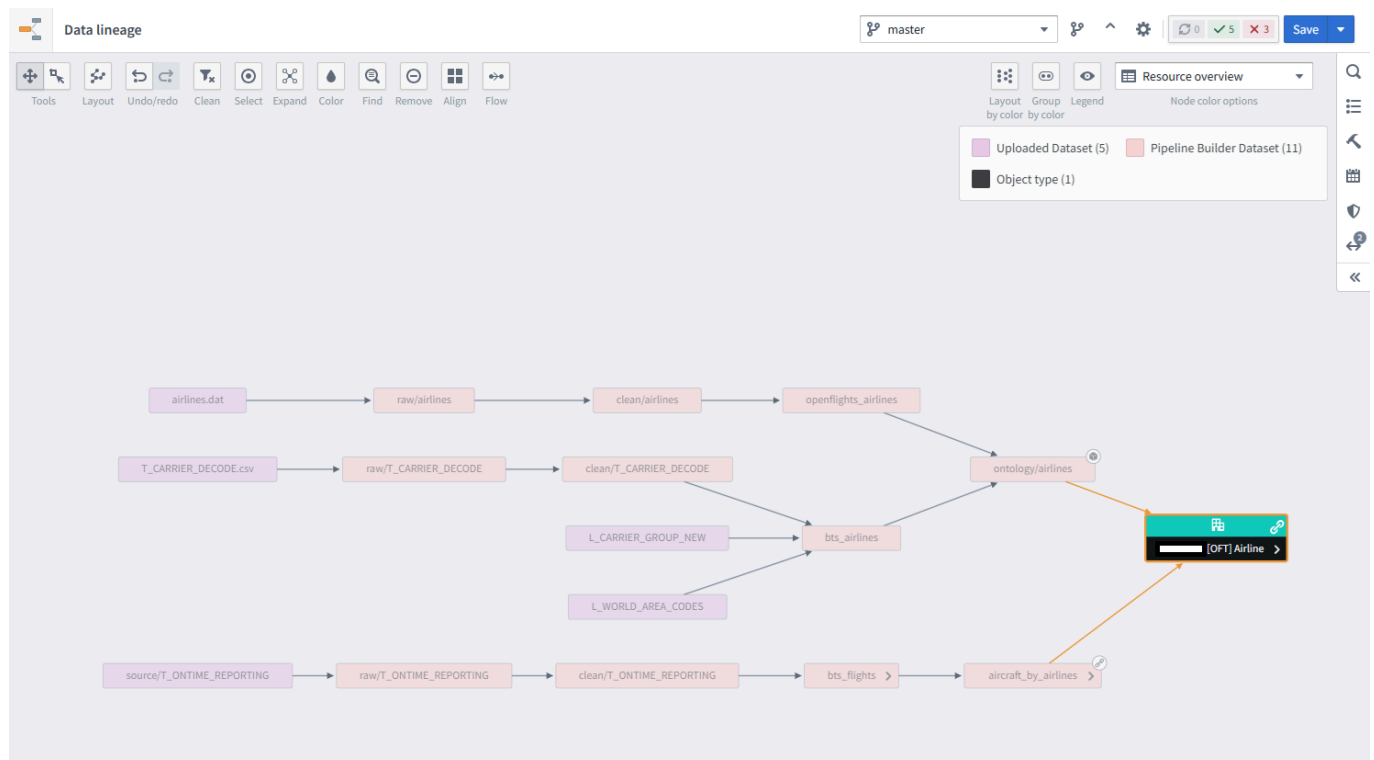
Data Lineage: Fresh Air pipeline

Lesson 9 / 20 · [原文链接](#)

Step 1: Explore the data lineage of an Airline

1. In Object Explorer, open the **Fresh Air Inc** object
2. In the top right, select **More > Advanced > Explore data lineage**

You should then find yourself in the Data Lineage application:



Step 2: Add other [OFT] object types

1. In the right side panel, switch to the **Search Foundry** tab (which has a magnifying glass icon)
2. Select **Object types**
3. If prompted, select your ontology
4. Filter by **[OFT]**
5. Select each of the 7 object types to add them to your graph - Airline, Aircraft, Route, Flight, Airport, Runway and Flight Alert.

Step 3: Expand all nodes

1. Select all nodes on your graph.

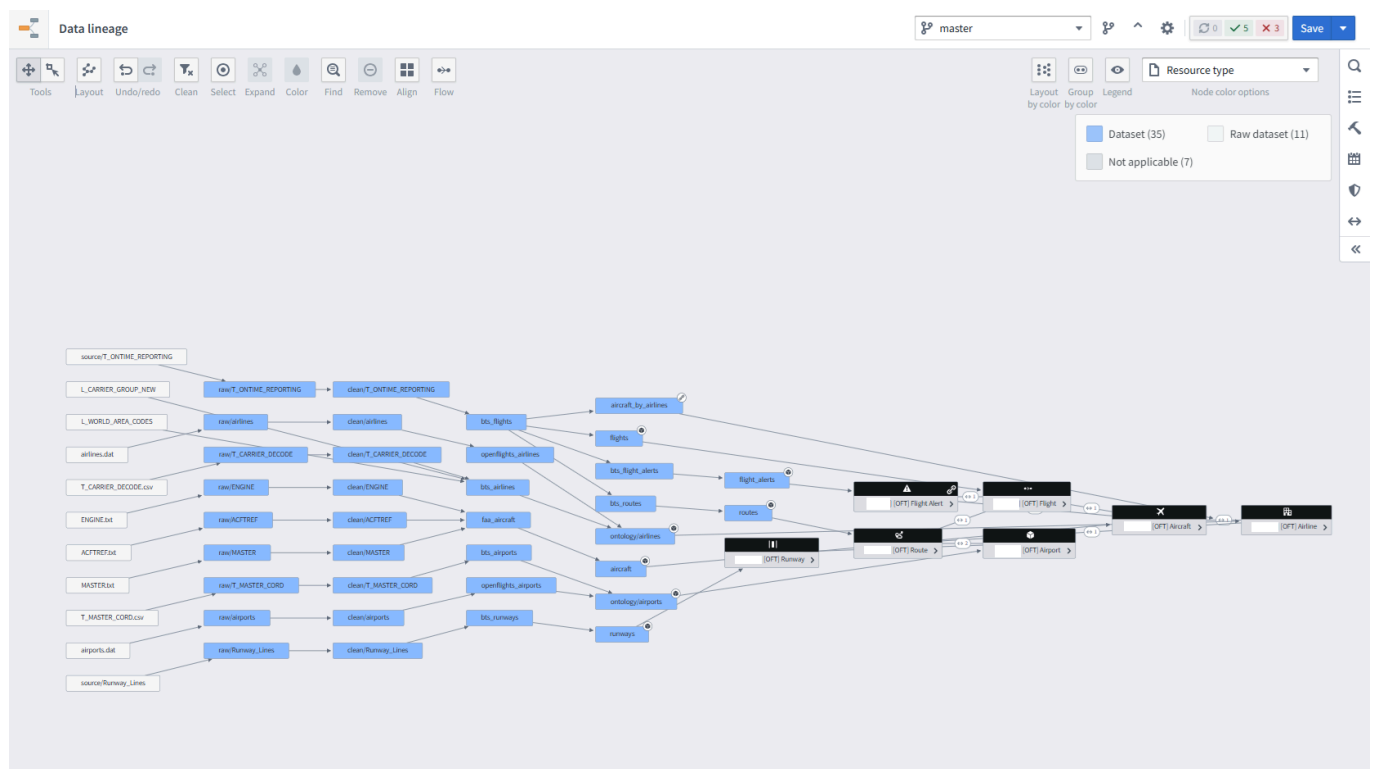
1. You can do this manually with your mouse by holding Shift and dragging a box over all nodes or you can press Ctrl+A / Cmd+A your keyboard
2. Click the **Expand** button in the top toolbar
3. Click the << button to **Expand parents**
4. Click **Add 30 nodes**

Step 4: Arrange all nodes

1. Select all nodes again
2. Click the **Layout** button in the top toolbar
3. Click **Layout all nodes**

You can also reposition nodes manually.

At this point, you can see that you have 11 uploaded (raw input) datasets that produce a pipeline of 35 other datasets, leading to 7 object types.



Data Lineage Exploration

Lesson 10 / 20 · [原文链接](#)

In the top right of the Data lineage graph is a **Node color options** drop-down. It defaults to **Resource overview** for new graphs. You will need to change that to **Resource type** within the drop-down to answer the following questions:

Are all of these datasets located in the same project?

Hint:

Answer:

Were all of these datasets produced by logic or code found in the same repository?

Hint:

Answer:

There is a tool panel found at the bottom of the screen and another on the right. Use those two panels to answer these questions:

How many rows and columns make up the ontology/airports dataset that backs the [OFT] Airport object type?

Hint:

Answer:

Can you view the logic that was used to derive the ontology/airports dataset? What application was used to define that logic?

Hint:

Answer:

This only scratches the surface of what you can learn about a pipeline's state using [Data Lineage](#), but it is sufficient for this exploration of Ontology.

Reflections

Lesson 11 / 20 · [原文链接](#)

You’ve now seen the many datasets that feed into the Fresh Air Ontology, what does this context mean for your understanding of the value of the Ontology?

Ease of Use

First of all, consider again the ease of use that the Ontology provides for data. Imagine answering the same questions you answered previously in the Object Explorer module but using only this graph of datasets and any dataset-based query or analysis tools. The first challenge would be deciding which datasets to start with and then how to combine them to produce a complete picture for any particular question. That work has already been done by the data engineer(s) who built this pipeline and the Ontology.

Common vocabulary

Secondly, we can see here how the Ontology provides a common vocabulary across an organization. For example, suppose the Fresh Air Flight Operations team used to use the Bureau of Transportation Statistics data as their canonical list of airports, while the Fleet Maintenance team used the list provided by OpenFlights. It is easy to have such discrepancies when working from your preferred dataset. However, given that there should really be only one Airport object type in the Ontology, this forces some discussion and resolution between these teams to establish a common ground truth.

This established common vocabulary then becomes the obvious destination for any significant future work. For example, any new datasets about airports should not be brought into Foundry and left as simply another available dataset. Instead, it should be merged into the Airport object type. (It may be that not every user should be able to see all details about every Airport object. However, it is possible in Foundry to apply both row and column-level [permissions](#) when defining the backing data sources for an object type so that you can still have a single Airport object type that presents only the data each user is allowed to see.)

Similarly, any significant analytical insights, useful aggregations, or model-derived recommendations can be added to the relevant object type. This makes this work centrally accessible to users across the organization, rather than languishing as forked or siloed local work.

Because object types can grow richer and more detailed like this over time, it becomes much easier to bootstrap new workflows and use cases that rely on the same object type. For example, once an Airport object type is created to support a Flight Operations workflow, that object type can be reused (and augmented) to build a new Fleet Maintenance workflow.

Some of the advantages of a common vocabulary could be achieved by simply marking certain datasets as the “golden tables” for a given concept. (For example, Foundry’s [Data Catalog](#) can help do this.) However, in practice, this does not prove to be as effective in controlling the entropy produced by diverse workflows. For example, as a user looking for the authoritative dataset on airports to use as a basis for a new workflow or analysis, you might find `airports`, but you’ll often find a set of datasets derived from that one with names like `airports_with_geojson`, `airports_fresh_air_staff_enriched`, and `better_airports`. While Data Lineage can help you understand the provenance of each of these datasets and how each of them have improved or modified the golden `airports` table, this work is effectively leading to local variants

and dialects in what is intended to be an authoritative common vocabulary. Because the Ontology provides a [object layer](#) separate from the dataset layer, it can provide stronger controls over these types.

Context: Extending Fresh Air's Ontology

Lesson 12 / 20 · [原文链接](#)

Now that you've seen the structure and some of the value of the Fresh Air Ontology, let's extend it.

As mentioned earlier, whenever a Flight is delayed, cancelled, or diverted, it would be useful to have an object to represent this situation. Each Flight Alert can then be responded to by Fresh Air's Flight Operations department. This is an example of an operational workflow in Foundry.

In this section, you'll build your own version of the Flight Alert object type. (You can use the existing version as a reference if you get stuck.) This will include a link to the relevant Flight and an action to update the root cause of the alert.

Ontology Manager

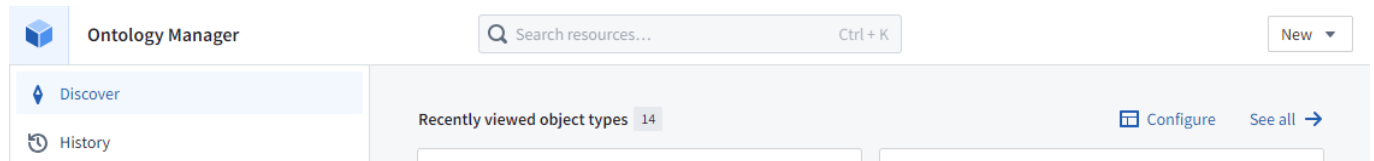
Lesson 13 / 20 · [原文链接](#)

Step 1: Open Ontology Manager

1. Open **Ontology Manager**

1. This can be found by searching the Applications menu found in the left side bar

You should then see an app header that looks like this:



[Ontology Manager](#) (sometimes called the Ontology Management Application or OMA) enables you to build and maintain your organization's Ontology.

While you can also define object and link types from Pipeline Builder (as you did in the End-to-End Speedrun course), Ontology Manager offers many more capabilities and options. Because you can currently (at the time of this writing) only edit ontology type definitions from the application you used to originally create it, it is generally recommended that you use Ontology Manager as your default ontology tool, unless you know that Pipeline Builder's lighter feature set will be sufficient for your needs.

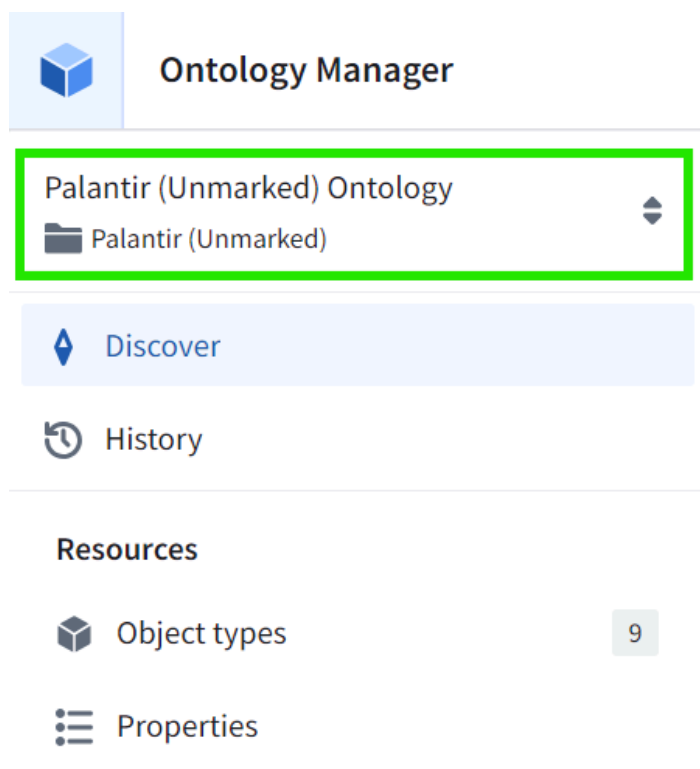
Create an object type

Lesson 14 / 20 · [原文链接](#)

Aside: Permissions

You may not have permission to create new object types in the default Ontology at your organization. If you run into an issue creating a new object type in the next step, see the *Permissions & Help* lesson early in this tutorial, the *Install the Marketplace Bundle* lesson, and/or try these two potential fixes:

1) Change the current Ontology. In the top left corner of the Ontology Manager application, there is a drop-down to select between different Ontologies (if more than one is available). If you explore this drop-down, you may find an Ontology that you have permission to edit and that contains the rest of the Fresh Air (OFT) Ontology.



2) Use Ontology Proposals. Some organizations limit edits to the production Ontology via [Ontology Proposals](#). If you are unable to create new Objects in an available Ontology, you may need to create a custom branch for the steps below and seek approval from your organization's Administrator for merging in the changes.

Step 1: Create Flight Alert

1. Click **New > Object type** in the top right corner of the screen
2. Select the **flight_alerts** dataset from the **Deep Dive: Creating Your First Ontology** folder as an existing datasource
 1. Click **Use existing datasource**

2. Click **Select datasource**
3. Search for the **flight_alerts** dataset found in you **Deep Dive: Creating Your First Ontology/data/flight alerts/ontology/** folder
4. Click **Next**
3. Name your object type **[<username>] Flight Alert** (where **<username>** is your own username)
 1. For example, if your username is jsmith, you would create **[jsmith] Flight Alert**
 2. The **Plural name** should update accordingly
 3. Click **Next**
4. Set the **Primary key** and **Title** properties
 1. Set **Primary key** to Flight Alert Id
 2. Set **Title** to Alert Title
 3. Click **Next**
5. Skip creating any generic action types
6. Click the green **Create** button

This should result in an overview of your new object type:

Ontology Manager

Discover

[username] Flight Alerts

0 objects

Overview

Properties 27

Security

Datasources

Capabilities

Object views

Interfaces

Materializations

Automations

Usage

History

Search resources... Ctrl + K

189 edits

Save

New

[username] Flight Alerts

Object type - 0 objects

Add to group

Plural name [username] Flight Alerts

Description Type here...

Aliases Add aliases...

Point of contact None

Contributors None

Ontology zacht Ontology

API name UsernameFlightAlerts

Status Experimental

Visibility Normal

Edits Disabled

ID username-flight-alerts

RID Set on save

Properties 27

New

Flight Alert Id Primary key

Alert Title Title

Root Cause

1.0 Priority

Action types 0

New

[username] Flight Alerts

Preview objects Preview table

	Flight Alert Id	Alert Title	Root Cause	Priority
	String	String	String	Double
1	2023-01-01-9E-5144-RDU-LGA-0745	Late Departure for 9E5144 (RDU-L...	null	62
2	2023-01-01-9E-5232-CHO-ATL-0600	Late Departure for 9E5232 (CHO-A...	null	388
3	2023-01-01-9E-5301-BTR-ATL-1659	Late Arrival for 9E5301 (BTR-ATL) ...	null	80
4	2023-01-01-9E-4815-ATL-BTR-0855	Diversion for 9E4815 (ATL-BTR) @ ...	null	1000
5	2023-01-01-9E-4815-BTR-ATL-1025	Late Departure for 9E4815 (BTR-A...	null	164
6	2023-01-01-9E-5025-ATL-SHV-1110	Late Departure for 9E5025 (ATL-S...	null	78

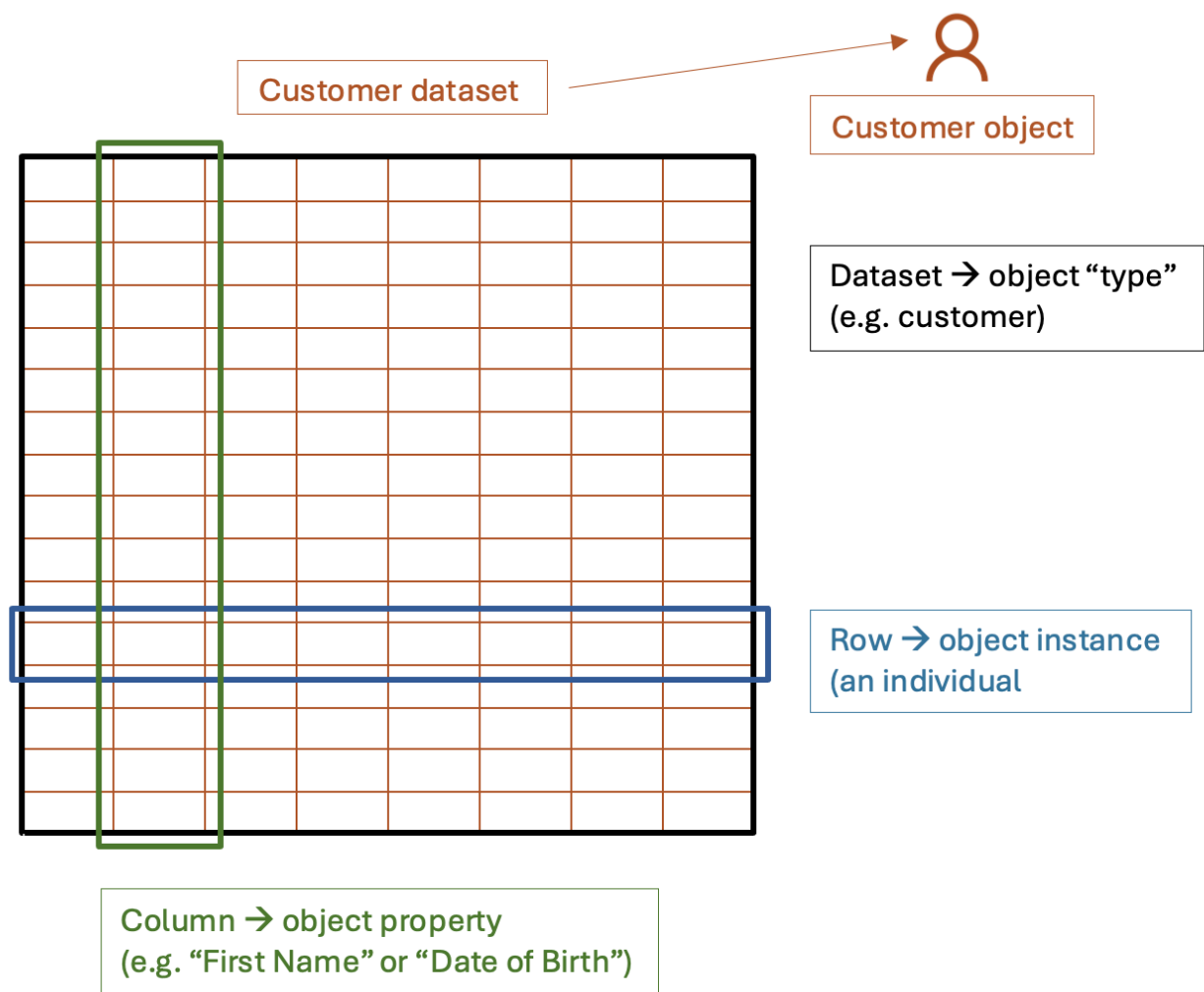
Concepts

Mapping a datasource to an object type

When you define a new **object type**, you are defining a kind of blueprint or template. This object type is backed by one or more datasources. This **datasource** is typically a dataset, but it could also be [restricted view](#), [virtual table](#), etc.

Each row of data in the backing datasource will produce one object that is a specific instance of that object type. Each column typically maps to a property. The permissions a user has to the backing datasource(s) also determines whether they can see the corresponding object data.

Here is example using Customers:



You can explore this row-to-object mapping in Ontology Manager. If you look at the bottom of your screen, you'll find a preview of the backing datasource. At the top of that panel, click **Preview objects** to preview the objects that correspond to each row.

By using actions, it is possible for users to create additional objects that do not exist in the backing datasource. Users can also delete objects. Therefore, over time, the mapping from rows to objects may not always exist for all rows.

Columns verses Properties

By default, when you create a new object type, each column of the backing datasource is mapped to a property in the new object type. However, you can change this mapping. For example, you can rename a property or choose not to map some columns to properties.

It is also possible to add properties that are not backed by a column. If you create a new property, you can set its *Source* to *User edits*. These values can only be created or changed later through actions on a per-object basis.

If you look through the preview of the **flights_alerts** dataset, you'll find that the `root_cause` column is empty (contains only null values). When initially building this pipeline, there was no satisfactory way to determine the `root_cause` for each alert from the other data available. Instead, we plan to collect this information from Flight Operations users as they investigate and close each alert. In this case, the Root Cause would have been a good candidate for an [edit-only](#) property without a backing column. However, creating an empty column as part of the data pipeline also works. This will make it easier to insert a data-derived root cause value later if we should ever integrate another data source that would let us update the data pipeline to determine at least some instances of this automatically.

The takeaway here is that, while datasource columns inform which object properties you have, they do not always map 1-to-1 because object data is separate from datasource data. (We'll explore this in a little more detail in Object Storage below.)

Primary Keys and Titles

Every object must have a unique identifier that is consistent for the life of that object. This is its **Primary key** property. When selecting a primary key, you must choose a column that will contain a unique value for each row and that is never null. Strings are typically the best data type to use for this.

If your backing datasource doesn't contain a column that will serve as a valid identifier, you might derive a new column that concatenates multiple columns (and optionally hashes the result) to generate a valid key. This must be a deterministic repeatable process. You should never generate a random ID or GUID for this purpose as part of a data pipeline, because, if the pipeline is ever rerun, all of your object IDs will change.

Every object should also have a **Title**. This property is used as the user-friendly and human-readable name of the object in search results and in most Foundry applications. While the primary key can also be used for this purpose, unique keys are often fairly unreadable or opaque to everyday users. Again, you might derive a useful title from other columns as part of your backing data pipeline.

Actions

Actions provide a controlled way to create, edit, and/or delete objects. Ontology Manager offers to create a generic version of these when you create a new object type.

In this example Fresh Air use case, we plan to only create new Flight Alerts through the data pipeline, so we don't need a *Create* action for users to do so manually. While we will resolve alerts, we won't delete them, so we also don't need a *Delete* action. We will allow users to edit an alert, but only by touching certain fields. So we will create a more targeted action for that purpose in a later step.

Step 2: Save your changes

1. Click **Save** in the top right of the screen
2. Click **Save to ontology**
3. Click **Save changes**

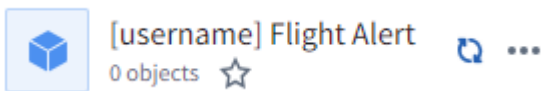
If other users have made changes to the ontology while you were working, you may be prompted to update your view of the Ontology to pull in their changes before you can save your own.

If you get any errors at this point, you can review the steps above to see if you made any mistakes in configuring the object type. If you get stuck, you can discard all of your edits and start over.

Concepts

Object Storage

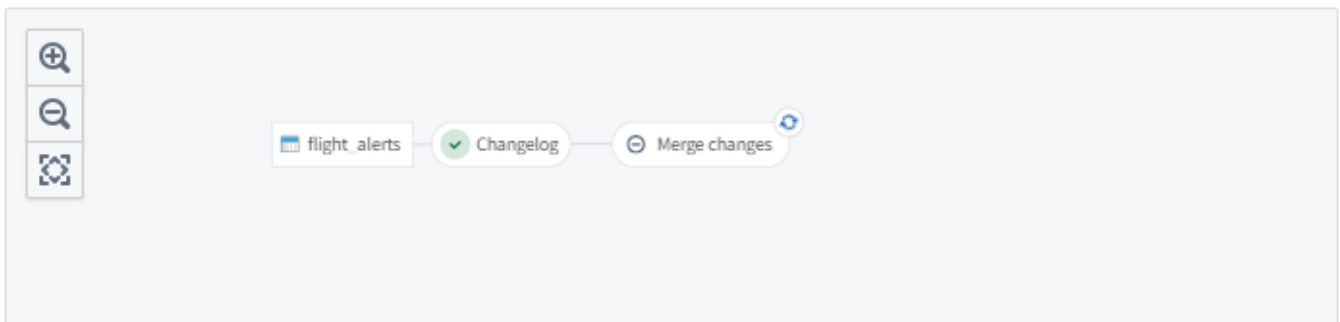
Once you save, you'll see a little progress spinner appear in the upper left of the screen:



This indicates that Foundry is currently building an index of all of your object data for this object type in [Object Storage v2](#). If you click this spinner, you can view the progress. This will take you the *Datasources* tab:

Replacement pipeline

Resync the data in the backing stores using the latest object type definition



If Foundry encounters any issues—such as non-unique primary keys—you'll be able to get to detailed job details and error messages from here.

Only once this indexing pipeline completes successfully will you be able to see your new objects in Object Explorer and other Foundry applications. Once it's done, you can refresh the page to see an object count in top left of your screen: *183, 999 objects*.

You do not need to wait for this pipeline to finish running to move on to the next step of this tutorial.

Create a link type

Lesson 15 / 20 · [原文链接](#)

It would be useful to be able to quickly navigate from each Flight Alert to its corresponding Flight object. There is a lot of ancillary information available on the Flight object that we don't need to repeat on the Flight Alert object, but that might be useful to users who are responding to an alert.

Each Flight Alert includes a Flight Id property that contains the primary key of the corresponding Flight.

Step 1: Create Flight Alert <> Flight link

1. Click **New > Link type** in the top right corner of the screen
2. Create a foreign key relationship type
 1. Select **Object type foreign keys**
 2. Click **Next**
3. Choose the two object types
 1. On the left: **[<username>] Flight Alert** (where <username> is your username)
 2. On the right: **[OFT] Flight**
4. Set the **Foreign key** for Flight Alert: Flight Id
 1. Click **Next**
5. Set **Cardinality**: **One** Flight to **Many** Flight Alerts
6. Click **Submit**

You should then have a link type definition that looks something like this:

Ontology Manager

View all related resourcesCtrl + K

1 editSaveNew

Discover

Flight Alerts

Flight Alert

[OFT] Flight

Overview

Security

Datasources

Usage

Status

Experimental

ID username-flight-alert-flight1

RID Set on save

Configuration

Join method

Foreign keyDataset

Choose the object types you wish to link, and the property to use as a foreign key.

Flight Alert

[OFT] Flight

Flight Id

Flight Id

Primary key

Flight Alert → [OFT] Flight

Each Flight Alert has one [OFT] Flight

API Name: !FlightAlert.oftFlight.get()

Visibility: Normal

[OFT] Flight → Flight Alert

Each [OFT] Flight has many Flight Alerts

API Name: oftFlight.FlightAlerts2.all()

Visibility: Normal

Concepts

Foreign key

Similar to the concept used in relational databases, a foreign key property in one object type contains values that appear as a primary key value in another object type.

Cardinality

This is another data modeling concept that is not specific to Foundry. The cardinality of a relationship indicates how many entities might be involved on the two sides of that relationship.

This will depend on your underlying data and how you are modeling the world. For example, if you are modelling the relationship between an Airline and its Aircraft as “present-day ownership”, it would be **one-to-many**: Each Airline might own many Aircraft, but each Aircraft is owned by only one Airline at a time.

On the other hand, if you are modeling ownership over time (where planes are sold from one airline to another) or operations (where partner airlines might use codeshares or an airline might lease aircraft to its subsidiary airlines), then the relationship would be **many-to-many**: An Airline has many Aircraft, while an Aircraft is operated by many Airlines. (The Airline-Aircraft link in the Fresh Air Ontology is many-to-many if you would like to see an example.)

With regards to the Flight Alert you just created, for this notional data pipeline, we generated only one Flight Alert per significantly delayed, cancelled, or diverted Flight. This would make this a practically **one-to-one** relationship. However, that is not a constraint we would want to impose here. Conceptually (and perhaps in a future version of this pipeline), we would expect this to be a **one-to-many** relationship: While each Flight Alert should only be related to one Flight, a Flight might generate multiple Alerts. For example a flight could depart late and then, several hours later, be redirected to a different airport. This would ideally produce two separate alerts.

Step 2: Save your changes

1. Click **Save** in the top right of the screen
 2. Click **Save to ontology**
 3. Click **Save changes**
-

Create an action type

Lesson 16 / 20 · [原文链接](#)

There's a lot of different decisions the Flight Operations team might take in response to a Flight Alert. For example, they might need to reassign gate crews, contact the maintenance team, arrange for a plane swap, etc.

A full cross-team workflow like this often needs to be implemented incrementally in Foundry. We'll assume we're just getting started with only the Flight Operations team for now. Their immediate need is to understand more concretely what is causing their delays. While root-causing a delay is still largely a manual process for them, capturing the conclusion in Foundry is a good first step. Let's add an action to do that.

Step 1: Enable edits

1. Open your **[username] Flight Alert** object type in Ontology Manager
2. Switch to the **Datasources** tab
3. In the **Edits** section, toggle on **Allow edits**

The screenshot shows the Ontology Manager interface. On the left is a sidebar with navigation links: Home, [username] Flight Alert (183,999 objects), Overview, Properties (27), Security, Datasources (highlighted), Capabilities, Object views, Interfaces, Materializations, Automations, Usage, and History. The main panel has a top bar with a search icon, 'View all related resources' (Ctrl + K), '1 edit', a green 'Save' button, and a 'New' dropdown. Below the top bar, the 'Backing datasource' section shows a configuration for 'flight_alerts' with a 'Replace' button and an 'Add new backing datasource' button. The 'Edits' section is expanded, showing three settings: 'Allow edits' (a green dot and a toggle switch highlighted with a green box), 'Track user edit history' (a toggle switch), and 'Conflict resolution' (a text description and a list of properties for 'flight_alerts').

Step 2: Create a new action type

1. Click **New > Action type** in the top right of the screen
2. Create a Modify object action for your Flight Alert
 1. Select **Object** from among the available tabs (default value)
 2. Select **Object type**: Your **[username] Flight Alert**
 3. Select **Modify object(s)**
 4. Click **Next**
3. Add a Root Cause property
 1. Select **Add property > Root Cause**
 2. Click **Next**
4. Name your action type
 1. Set **Action type name**: **[<username>] Assign root cause** (where **<username>** is your username)
 2. Click **Next**
5. Give yourself permission to execute this action
 1. Switch to the **User** tab
 2. User is: (yourself)
6. Click **Create**

You should then see the definition of your new action:

The screenshot displays the 'Ontology Manager' interface. On the left, a sidebar contains navigation options: 'Discover', '[username] Assign root cause' (with a pencil icon and a green checkmark), 'Overview' (highlighted with a blue bar), 'Rules', 'Form', 'Capabilities', 'Security & Submission Criteria', and 'Automations'. The main area is titled 'Ontology Manager' and includes a search bar with the text 'View all related resources' and a 'Ctrl + K' shortcut. To the right of the search bar, it shows '22 edits', a green 'Save' button, and a 'New' dropdown menu. The main content area is divided into two sections. The top section, titled 'Name', shows '[username] Assign root cause' and a 'Status' dropdown set to 'Experimental'. Below this, the 'Description' field is empty with the placeholder 'Type here...'. The bottom section, titled 'API', shows 'username-assign-root-cause' and a 'RID' of 'ea08db6e-7355-d751-a37a-cc4a24ca4ebb'. The bottom section also includes an 'Action type overview' diagram. This diagram shows an 'Input' box with '[username] Flight Alert' and 'Root Cause', and a 'Rules' box with 'Modify object' and 'Modify Root Cause'. The 'Rules' box is connected to the 'Input' box by a line.

Concepts

Actions provide a controlled way to update objects. Because we've only defined one action that touches only the Root Cause property, this is the only property that users can edit.

Action Form

Every time you invoke this action from different contexts in Foundry, a common input form will collect the necessary input from a user. If you switch to the **Parameters** tab of your action type, you can see a preview of this form.

There is a lot of further customization you could do here. For example, you can rename input fields, assign default values, hide fields, and more. For example, for now we are going to accept any free text as the new Root Cause value. If you wanted to, you could constrain this to a list of multiple choices that you either enter manually or derive from a dataset. (You can see the original *[OFT] Assign root cause* action type's Root Cause parameter for an example of this.)

This customization is another example of how actions can control changes to data: You might limit choices to only certain allowed input values.

Submission Criteria

Another control is to specify who can invoke the action at all. You set this during action type creation. You can update it on the **Security & Submission Criteria** tab.

Rules, Side-effects, & Functions

When you execute an action, it runs one or more rules, which you can see on the **Rules** tab. For now, we are applying a very simple *Modify object* rule that sets one property value. However, you could add other rules that send a Foundry notification to another Foundry user, invoke a webhook to call out to an external system, or create and delete other objects.

Instead of these rules, you can instead back your action [with a function](#). Such code-based [functions](#) are also used elsewhere in Foundry beyond just backing actions.

Step 3: Save your changes

1. Click **Save** in the top right of the screen
 2. Click **Save to ontology**
 3. Click **Save changes**
-

Execute your action

Lesson 17 / 20 · [原文链接](#)

Let’s confirm that your new action works.

Step 1: Assign a root cause to a flight alert

1. Open **Object Explorer**
2. Search for your **[<username>] Flight Alert** object type
 - This should open an exploration of all your Flight Alerts
3. Click on one Flight Alert object in the **Results** list on the right
 - This will open a single Flight Alert object. You should see its many properties and a link to the corresponding Flight object.
4. In the top right, select **Actions > [username] Assign root cause**
 - If you don’t see the Actions menu, make sure you’ve saved your ontology changes and it has finished indexing those changes. If you already had this Object Explorer session open before making those changes, refresh the page to ensure it is in sync with the latest Ontology.

[username] Fligh... xLate Arrival for H... xNew explorationExplorations ▾Lists ▾

Late Arrival for HA29 (SEA-OGG) @ 2023-02-19T0950Experimental ☆[username] Flight Alert

🔄💬🔍⚡ Actions ▾More ▾

☰ Properties

Airline Route Id	HA-SEA-OGG	Alert Reason	late arrival
Alert Title	Late Arrival for HA29 (SEA-OGG) @...	Arrival Delay	45
Cancellation Reason	No value	Cancelled	No
Carrier Delay	45	Day Of The Week	Sunday
Departure Delay	-1	Dest	OGG
Diverted	No	Flight Alert Id	2023-02-19-HA-29-SEA-OGG-0950
Flight Date	Feb 19, 2023	Flight Id	2023-02-19-HA-29-SEA-OGG-0950
Flight Number	HA29	Flight Title	HA29 (SEA-OGG) @ 2023-02-19T0...
Late Aircraft Delay	0	Nas Delay	0
Operating Carrier Id Unique	HA	Origin	SEA
Priority	90	Root Cause	No value
Route	SEA-OGG	Scheduled Departure Time	0950
Security Delay	0	Tail Number	N379HA
Weather Delay	0		

🔗 Links

[OFT] Flight 1

> HA29 (SEA-OGG) @ 2023-02-19T0950

[username] Assign root cause

5. Enter an example root cause

1. Such as: Waiting for gate assignment upon arrival

2. Click **Submit**

You should see your change reflected on in the Root Cause property of the object view.

Reflections

Lesson 18 / 20 · [原文链接](#)

The action you just defined here is very basic, recording only a single Root Cause value. However, as you saw, this action has a semantic meaning, and it provides strong controls on who can make what kinds of changes to Ontology data. It also captures a single human conclusion back into Foundry.

If we were to continue build out this workflow for the Flight Operations teams, you can imagine other **decisions** and the resulting actions they might want to make:

- Assign an incoming Flight Alert to a given Flight Operations operator
- Forward the alert to the local Flight Maintenance team for processing
- Reassign the flight crew
- Queue for passenger rebooking
- Update the gate assignment
- Reschedule this aircraft's next flight
- Swap aircraft for this aircraft's next scheduled flight
- Make as resolved

As you focus on these decisions that the Flight Operations team needs to make, it shines a light on what additional data they will need in the Ontology to inform those decisions:

- Flight crew status and availability
- Plane availability for swaps
- Passenger counts and destinations to inform tradeoffs between plane swaps and rebooking

As this workflow grows in complexity, the Flight Operations team would quickly leave Object Explorer behind. Custom Workshop applications—as you built in the earlier End-to-End Speedrun course—would provide more appropriate and targeted tools. But those Workshop applications would reuse everything you built here: data pipeline, object type, links, and actions.

This illustrates another value of the Ontology. By focusing on real-world concepts, tailored to a specific organization and reflecting its processes, it lets us build solutions around daily decisions that need to be made. This produces a dynamic “digital twin” of the organization.

Conclusion

Lesson 19 / 20 · [原文链接](#)

This course has covered the basics you need to know to get started with a Foundry Ontology.

Value

We covered why the Ontology is valuable.

1. It makes data easier to use.
2. It structures data into a vocabulary that is reusable across workflows and across the organization.
3. The Ontology provides well-defined actions that can enforce constraints and make the output of regular decisions immediately to others within the platform.

In this way, the Ontology is more than another “semantic” data layer. It also includes “logic” through the rules that it encodes and “kinetics” through its actions. It is both the nouns of your organization, but also the verbs.

Ontology and AI

One last reason that the Ontology is valuable is the way that it allows you to integrate AI into daily operations.

First of all, the Ontology can provide clean and usable inputs to AI. For example, if we wanted AI-generated recommendations for how to respond to a given Flight Alert, it would need to review much of the same data that human-operators also need to power their decisions. Furthermore, the Ontology structures these concepts in real-world terms that an LLM are already familiar with.

Secondly, the Ontology provides a destination for AI outputs. The best place for a Flight Alert recommendation or prediction would be on the Flight Alert object itself. This inserts the recommendation into one or more existing operational workflows at the point where operators are already making their decisions. This makes it easier to then collect feedback on the final decisions those human operators then make—when they use the recommendations and when and why they disregard them.

The many tools of AIP aim to do exactly this by building on top of the Ontology.

Tools

In terms of specific tools, you learned about:

- [Object Explorer](#) - For object searches and for basic exploration and analysis across sets of objects
- [Data Lineage](#) - For exploring data pipelines and understanding how they lead to the Ontology
- [Ontology Manager](#) - For building the Ontology itself

Next Steps

To further your understanding of how objects can be tailored to support workflows directly:

- [Object Views](#) - The Airline and Airport objects you explored in this tutorial had custom object views

Good next steps to see how objects are then used by other applications:

- [Quiver](#) - For powerful point-and-click analysis of object sets and dashboarding
- [Workshop](#) - For building operational applications from point-and-click widgets
- [Ontology SDK \(OSDK\)](#) - Building custom third-party applications that treat the Ontology as a backend

Loading...

Lesson 20 / 20 · [原文链接](#)

Loading...